- **Lab Outcomes:**
- **Understand Array Basics**: Learn how to declare and initialize arrays in C#, and understand the concept of arrays as reference types stored at contiguous memory locations.
- **Array Operations**: Use built-in array methods such as Max(), Min(), Sum(), Average(), Sort(), and Reverse() to manipulate and process array data.
- **Array and Functions**: Learn how to pass arrays to functions in C# for better code reuse and modularization.
- **Multidimensional Arrays**: Understand how to create and work with multidimensional arrays (e.g., 2D or 3D arrays) in C#, and use them for more complex data storage and retrieval.
- **Understand Jagged Arrays:** Learn the concept of jagged arrays, where each array element can have a different size, and understand how they differ from multidimensional arrays.

C# Array

An array stores a fixed-size sequential collection of elements of the same type. An array is used to store a collection of data, but it is often more useful to think of an array as a collection of variables of the same type stored at contiguous memory locations.

Declaring Array

To declare an array in C#, you can use the following syntax

```
datatype[] arrayName;
```

Where,

- *datatype* is used to specify the type of elements in the array.
- *[ ]* specifies the rank of the array. The rank specifies the size of the array.
- *arrayName* specifies the name of the array.

Initializing an Array

Declaring an array does not initialize the array in the memory. When the array variable is initialized, you can assign values to the array.

Array is a reference type, so you need to use the new keyword to create an instance of the array. For example,

```
double[] balance = new double[10];
```

C# Array Method

- Max()
- Min()
- Sum()
- Average()
- Sort()
- Reverse()

C# Passing Array to function

In C#, to reuse the array logic, we can create function. To pass array to function in C#, we need to provide only array name.

```
functionname(arrayname);//passing array
```

Example

```
static void printArray(int[] arr)
```

```
{
    Console.WriteLine("Printing array elements:");
    for (int i = 0; i < arr.Length; i++)
    {
        Console.WriteLine(arr[i]);
    }
}
public static void Main(string[] args)
{
    int[] arr1 = { 25, 10, 20, 15, 40, 50 };
    int[] arr2 = { 12, 23, 44, 11, 54 };
    printArray(arr1);//passing array to function
    printArray(arr2);
}
```

C# Multidimensional Array

The multidimensional array is also known as rectangular arrays in C#. It can be two dimensional or three dimensional. The data is stored in tabular form (row * column) which is also known as matrix.

To create multidimensional array, we need to use comma inside the square brackets. For example:

```
int[,] arr=new int[3,3];//declaration of 2D array
int[,,] arr=new int[3,3,3];//declaration of 3D array
```

C# Jagged Array

Jagged array is an **array of arrays** such that member arrays can be of different sizes. In other words, the length of each array index can differ. The elements of Jagged Array are reference types and initialized to null by default.

Declaring Array

In Jagged arrays, user has to provide the number of rows only. If the user is also going to provide the number of columns, then this array will be no more Jagged Array.

```
data_type[][] name_of_array = new data_type[rows][]
```

Example:

```
int[][] jagged_arr = new int[4][]
```

Initializing a jagged Array

The elements of Jagged Array **must be initialized** before its use. You can separately initialize each array element. There are many ways to initialize the Jagged array's element.

```
jagged_arr[0] = new int[2];
jagged_arr[1] = new int[4];
jagged_arr[2] = new int[6];
jagged_arr[3] = new int[7];
```

Another method:

```
jagged_arr[0] = new int[] {1, 2, 3, 4};
jagged_arr[1] = new int[] {11, 34, 67};
jagged_arr[2] = new int[] {89, 23};
jagged_arr[3] = new int[] {0, 45, 78, 53, 99};
```

Example

```
public static void Main()
{

   // Declare the Jagged Array of four elements:
   int[][] jagged_arr = new int[4][];

   // Initialize the elements
   jagged_arr[0] = new int[] {1, 2, 3, 4};
   jagged_arr[1] = new int[] {11, 34, 67};
   jagged_arr[2] = new int[] {89, 23};
   jagged_arr[3] = new int[] {0, 45, 78, 53, 99};

   // Display the array elements:
   for (int n = 0; n < jagged_arr.Length; n++) {

      // Print the row number
      System.Console.Write("Row({0}): ", n);

      for (int k = 0; k < jagged_arr[n].Length; k++) {

         // Print the elements in the row
         System.Console.Write("{0} ", jagged_arr[n][k]);
      }
      System.Console.WriteLine();
   }
}
```

C# Jagged array with Multidimensional Array

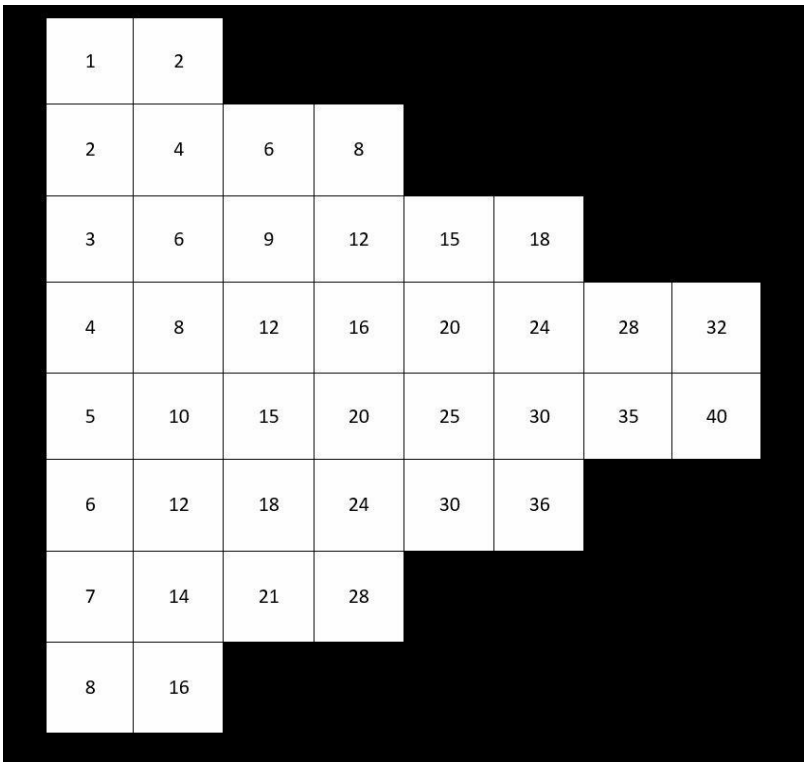
It is possible to mix jagged and multidimensional arrays. Below are the declaration and initialization of a 1-D jagged array that contains four two-dimensional array elements of different sizes.

```
int[][, ] jagged_arr1 = new int[4][, ]
{
    new int[, ] { {1, 3}, {5, 7} },
    new int[, ] { {0, 2}, {4, 6}, {8, 10} },
    new int[, ] { {7, 8}, {3, 1}, {0, 6} },
    new int[, ] { {11, 22}, {99, 88}, {0, 9} }
};
```

Tasks:

Task 1: Write a program that inputs the marks of 3 students in three different subjects. If student is passed in two subjects then he will be considered pass and also displays the number of students who fail.

Task 2: Implement the following:



Task 3: Abdullah has offered his teacher a treat including his friends Ali and Sheraz under a condition that each will order exactly three dishes of any cost. After bill has received Abdullah want to calculate the total cost of each individual's dishes. Help him by making a program:

a. Write single functions that return the bill of each participant.
b. Tell him, who is the most costly person.

Lab Assessment Criteria:
Lab #04 Marks distribution

	ER1	ER2	ER8
Task	3 points	3 points	4 points

Lab # 04
Rubric Evaluation Guideline:

#	**Qualities & Criteria**	**0 < Poor <=1**	**1.1 < Satisfactory <2**	**2.1 < Excellent <=3**
ER1	Task/Program Completion	Minimal or no task functionality was achieved	Some tasks were completed but few has errors or incomplete functionalities.	All tasks were completed without any errors.
#	**Qualities & Criteria**	**0 < Poor <=1**	**1.1 < Satisfactory <= 2**	**2.1 < Excellent <=3**
ER2	Code Structure and Design	Poor code structure and missing essential code elements like functions, data structures, or control flow.	Code functions but lacks proper structure, documentation, or uses inefficient algorithms.	Well-structured, documented code using appropriate data structures and algorithms.
#	**Qualities & Criteria**	**0 < Poor <= 1.5**	**1.6< Satisfactory <= 3**	**3.1< Excellent <= 4**
ER8	Question & Answer	Answers some questions but not confidently or based on lab task knowledge.	Answers most questions confidently and based on lab task knowledge.	Answers all questions confidently and demonstrates a deep understanding of the given lab task.